Research Interests and Directions

Tommaso Cerruti

Correspondence: tcerruti@ethz.ch

Research vision

AI research has the peculiarity of invalidating the methods we use to study it as it progresses. As models become more capable and are embedded into increasingly complex systems, assumptions that used to seem reasonable can (silently) stop holding: an evaluation may no longer measure the capability it was designed to capture, a security boundary may disappear once information flows through harnesses, and a computational mechanism may become limiting as the scale and nature of the system change. I am interested in understanding when these transitions happen and in developing methods that remain reliable as the systems around them evolve, motivating my work on **robust evaluation**, **agent security**, and **neural computation**.

1 Robust Evaluation

When we evaluate an agent, we do not directly measure its capabilities; we measure its performance on tasks designed to approximate those capabilities. Designing evaluation artifacts that allow fair evaluation of agents, while stress-testing them at the frontier of their capabilities, is therefore one of the most complex engineering problems in current AI research, particularly given how capable modern agents have become.

Recent work from OpenAI has highlighted the problem of flaky benchmarks, where evaluation artifacts can be mistaken for model failures or capability gains (OpenAI 2026b). This becomes particularly important for realistic agent evaluations: agents can take unconventional but valid routes to solve a problem, and an evaluation should not unnecessarily constrain these trajectories (unless the route itself is what the benchmark intends to study). Terminal-Bench 4.0 (Marten, . . . , **Cerruti**, 2026) aims to provide rigorous evaluation of frontier agents on realistic terminal tasks while preserving flexibility in how those tasks are solved, and was featured in the launch evaluations for GPT-6 Astra, Claude Fable 5.1 and Mythos 5.1, and Gemini 3.8 Flash (OpenAI 2026a; Anthropic 2026; Google 2026). CocoaBench (CocoaBench Team, . . . , **Cerruti**, 2026) instead focuses on compositional evaluation, testing whether capabilities that models exhibit independently can be combined successfully within realistic digital tasks.

Beyond benchmark construction itself, evaluation scores are tightly tied to their experimental protocol, which can materially shape the behavior we observe. Inference-time compute can substantially change measured capability, as discussed in *Safety Evals Should Project Test-Time Compute* (**Cerruti** 2026), later cited and studied empirically in McFadyen et al. (2026). Agent reliability can also differ substantially under repeated execution or perturbations of the evaluation environment (Rabanser et al. 2026). More generally, evaluation results are inseparable from choices such as compute budget, hardware, scaffolding, harnessing, and other experimental details, motivating work on evaluation standardization and reproducibility, such as Every Eval Ever (Batzner, . . . , **Cerruti**, 2026), which aims to make evaluation results more directly comparable.

Overall, robust evaluation requires rigorous frameworks for constructing tasks, validating evaluation artifacts, specifying experimental conditions, and interpreting the resulting capability signal. I believe there remains substantial room for improving how reliably our evaluations capture what increasingly capable agents can do.

Research directions

1. Safety under inference scaling. Do useful and unsafe capabilities scale at the same rate as agents receive more inference-time compute? Additional reasoning budget may help agents solve harder tasks and better understand constraints, but it may also enable more effective reward hacking, authorization violations, or oversight evasion. I would vary reasoning effort, token budgets, or parallel attempts on safety-relevant agent tasks, and estimate how useful capability and different unsafe behaviors scale with compute. The key question is whether safety improves alongside capability, or whether they follow different scaling patterns.

2. State-conditioned agent evaluation. What determines performance differences between agents in interactive benchmarks: the states they reach, or how well they act from those states? In long-horizon

environments, early actions shape the situations encountered later, so end-to-end scores combine trajectory generation with state-conditioned competence. I would snapshot intermediate states reached by different agents and evaluate all models from a shared set of these states. The key question is how much of the performance gap between agents comes from reaching different states, rather than behaving differently when faced with the same ones.

2 Agent Security

As models become persistent tool-using agents, security is no longer only a property of the model or its current prompt: memory, harnesses, tools, delegation, identity, provenance, and interactions with other agents, all become part of the security perimeter.

In *Agent Memory Is a Surface for Endogenous Authorization Laundering* (**Cerruti** et al. 2026a), ordinary memory writing and summarization can accidentally broaden the authorization conveyed by previous interactions, causing a downstream agent to take actions that were never authorized. Importantly, this is not an adverserial failure: it emerges from the normal operation of the agent itself endogenously.

Existing work shows related failures across the agentic stack: persistent memory can be poisoned by adversarial inputs (Dash et al. 2026); memory transformations can lose the provenance of critical information (Xu et al. 2026); and agents can take harmful actions under goal conflict or pressure (Lynch et al. 2025). As agents increasingly use reusable artifacts, these abstractions become part of the security boundary (Pan et al. 2026).

Research directions

1. Endogenous epistemic privilege escalation. Can an agent turn its own uncertain beliefs into trusted facts? A model may infer something tentatively, encode it in memory (a report, database, file, or similar), and later cause that artifact to be treated as stronger evidence than the original inference. I would construct controlled agent workflows in which the same uncertain information is externalized through different channels, then measure how downstream agents update their confidence and actions. The key question is whether information gains epistemic authority just by passing through an agent-generated artifact, and whether preserving provenance or uncertainty prevents this amplification.

2. Capability laundering through agent-generated abstractions. Can security-relevant effects become less visible when an agent packages permitted actions into a higher-level abstraction? Tool-using agents can combine primitive actions into scripts, helper tools, or reusable skills that are later invoked as a single operation, potentially hiding how the resulting capability is composed. I would construct workflows in which the same behavior is executed either directly through primitive tools or through agent-generated abstractions, while varying abstraction depth, persistence, permissions, and how much of the underlying implementation is visible to downstream agents or monitors. The key question is whether abstraction can weaken security boundaries even when no individual step introduces a new permission.

3 Neural Computation

Modern neural models learn powerful computational mechanisms, and understanding them can suggest better ways to build them. Attention heads can sometimes be approximated and replaced by executable programs (Hayes et al. 2026), making their computation explicit and directly manipulable. Linear-attention architectures instead replace standard attention with structured recurrent memory updates (Kimi Team et al. 2025; Hatamizadeh et al. 2026), offering a state-based view of how information is stored and transformed over a sequence. *Linear Attention Architectures* (**Cerruti** et al. 2026b) studies recent mechanisms through this common recurrent-memory view and explores how their states can be routed across layers.

Beyond interpretability, understanding neural computation may let us restructure it into simpler or more efficient forms. Recent work suggests that independently trained networks solving the same task can develop similar internal organization (Krauss et al. 2026), but do they also learn similar computations? If these computations can be expressed as executable programs, we can compare them directly across architectures and ask whether different models learn different algorithms or just different implementations of the same one.

Research directions

1. Algorithmic convergence across neural architectures. When different neural architectures learn the same task, do they discover the same underlying algorithm? Standard attention and recurrent linear attention process information in very different ways, yet they may still converge to computationally equivalent solutions. I would train both architectures on the same controlled tasks, keeping their data, optimization, model size, and training budget as comparable as possible. I would then use program synthesis or other executable representations to express and compare their learned computations in a common form. The key question is whether architecture mainly changes how an algorithm is implemented, or whether it guides learning toward different algorithms (e.g., finding the largest value by tracking the current max or by sorting the sequence).

2. Computational crystallization during learning. At what point during training does neural computation become describable as a stable explicit program? Existing program-synthesis methods typically study computations only after they have been learned (Hayes et al. 2026); I am interested instead in how programmatic structure emerges during training. I would apply program synthesis across model checkpoints, tracking when the resulting programs become faithful, simple, stable, and able to replace the original neural computation. The key question is whether these changes correspond to changes in generalization, and what happens if a crystallized neural component is replaced by its program while training continues.

References

Anthropic (Sept. 2026). *Introducing Claude Fable 5.1 and Claude Mythos 5.1*. Model announcement.

Batzner, J., S. H. Nelaturu, ..., T. **Cerruti**, et al. (2026). *Every Eval Ever: A Unifying Schema and Community Repository for AI Evaluation Results*. arXiv: 2606.14516 [cs.AI].

Cerruti, T. (May 2026). *Safety Evals Should Project Test-Time Compute*. Hugging Face Community Article.

Cerruti, T., M. Okamoto, and A. K. Erol (2026a). *Agent Memory Is a Surface for Endogenous Authorization Laundering*. arXiv: 2609.01836 [cs.CR].

Cerruti, T., T. Rieder, G. Rowlands, L. Jin, and I. Schlag (2026b). *Linear Attention Architectures: Mechanisms, Trade-offs, and Cross-Layer Routing*. arXiv: 2607.07953 [cs.LG].

CocoaBench Team, S. Hao, ..., T. **Cerruti**, et al. (2026). *CocoaBench: Evaluating Unified Digital Agents in the Wild*. arXiv: 2604.11201 [cs.AI].

Dash, P., T. Ge, A. Jain, T. Shah, and Z. Shang (2026). *From Untrusted Input to Trusted Memory: A Systematic Study of Memory Poisoning Attacks in LLM Agents*. arXiv: 2606.04329 [cs.AI].

Google (Sept. 2026). *Introducing Gemini 3.8 Flash and 3.8 Flash Cyber*. Model announcement.

Hatamizadeh, A., Y. Choi, and J. Kautz (2026). *Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention*. arXiv: 2605.22791 [cs.LG].

Hayes, A., B. Li, and J. Andreas (2026). *Explaining Attention with Program Synthesis*. arXiv: 2606.19317 [cs.LG].

Kimi Team et al. (2025). *Kimi Linear: An Expressive, Efficient Attention Architecture*. arXiv: 2510.26692 [cs.LG].

Krauss, P., A. Schilling, A. Maier, T. Kinfe, H. Stübner, N. Römmelt, and C. Metzner (2026). *Convergent Evolution in Algorithmic Space*. arXiv: 2608.05985 [q-bio.NC].

Lynch, A., B. Wright, C. Larson, S. J. Ritchie, S. Mindermann, E. Perez, K. K. Troy, and E. Hubinger (2025). *Agentic Misalignment: How LLMs Could Be Insider Threats*. arXiv: 2510.05179 [cs.AI].

Marten, R., A. Shaw, I. Bercovich, B. Droste, T. **Cerruti**, et al. (Aug. 2026). *Terminal-Bench*. Version v4.0.0.

McFadyen, J., O. Jorgensen, H. Coppock, K. Wei, and C. Ududec (2026). *How Inference Compute Shapes Frontier LLM Evaluation*. arXiv: 2606.17930 [cs.AI].

OpenAI (Sept. 2026a). *GPT-6 Astra: A New Generation of Intelligence*. Model announcement.

— (July 2026b). *Separating Signal from Noise in Coding Evaluations*. Research publication.

Pan, S., X. Sun, T. Zhang, D. Liao, M. Si, and Z. Xing (2026). *SkillGuard: A Permission Framework for Agent Skills*. arXiv: 2606.03024 [cs.CR].

Rabanser, S., S. Kapoor, P. Kirgis, K. Liu, S. Utpala, and A. Narayanan (2026). *Towards a Science of AI Agent Reliability*. arXiv: 2602.16666 [cs.AI].

Xu, J., Y. Xiao, W. Shao, H. Liu, and X. Li (2026). *Memory Provenance Laundering in LLM Agents: A Non-Amplification Firewall for Persistent Memory*. arXiv: 2607.29167 [cs.AI].